LAPORAN PRAKTIKUM

POSTTEST (5)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Fachrul Aulia Rahman (2509106030)

Kelas (A2 ‘25)

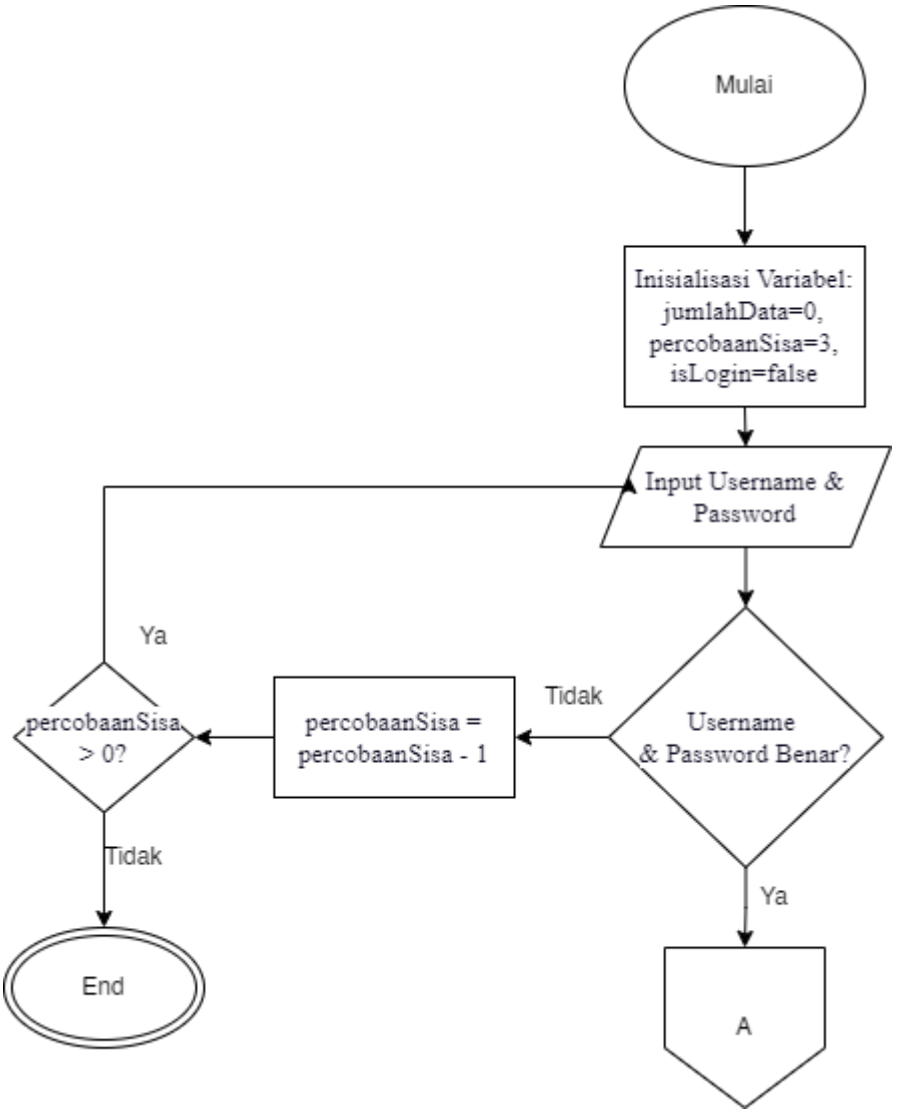
PROGRAM STUDI INFORMATIKA

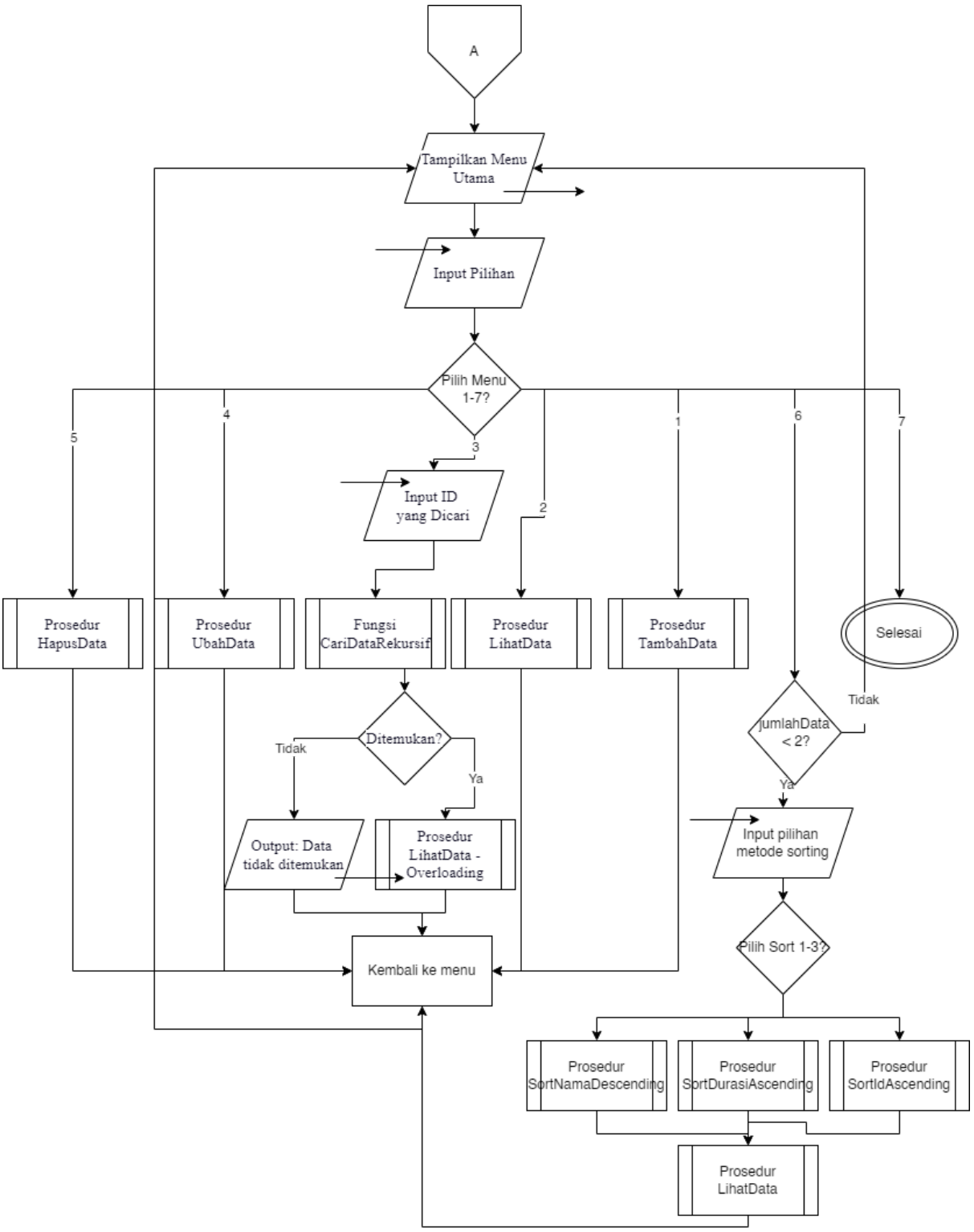
UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart


Mulai
Inisialisasi Variabel: jumlahData=0, percobaanSisa=3, isLogin=false
Input Username & Password
Username & Password Benar?
Tidak
percobaanSisa = percobaanSisa - 1
percobaanSisa > 0?
Ya
Tidak
End
Ya
A


A
Tampilkan Menu Utama
Input Pilihan
Pilih Menu 1-7?
5
4
3
2
1
6
7
Input ID yang Dicari
Prosedur HapusData
Prosedur UbahData
Fungsi CariDataRekursif
Prosedur LihatData
Prosedur TambahData
Selesai
Ditemukan?
Tidak
Ya
Output: Data tidak ditemukan
Prosedur LihatData - Overloading
Kembali ke menu
jumlahData < 2?
Tidak
Ya
Input pilihan metode sorting
Pilih Sort 1-3?
Prosedur SortNamaDescending
Prosedur SortDurasiAscending
Prosedur SortIdAscending
Prosedur LihatData

2. Deskripsi Singkat Program

Program ini adalah aplikasi untuk mencatat penyewaan power bank.

Berikut cara kerjanya:

1. Login Aman: Kamu harus memasukkan username (Fachrul) dan password (030) terlebih dahulu. Jika salah 3 kali, sistem akan mengunci dan berhenti otomatis agar tidak sembarangan diakses orang lain.

2. Kelola Data (CRUD): Setelah masuk, kamu bisa melakukan empat hal utama: • Tambah: Mencatat penyewa baru (nama, merek, dan durasi). • Lihat: Melihat daftar semua penyewa yang sedang aktif. • Ubah: Mengedit data penyewa jika ada kesalahan input. • Hapus: Menghapus data jika pelanggan sudah selesai menyewa.

3. Sistem Otomatis: Program akan terus berjalan (menu muncul terus) sampai kamu memilih untuk keluar. Data disimpan sementara di memori komputer selama program berjalan.

3. Source Code

A. Fitur Login

Fungsi: Mengamankan program agar hanya bisa diakses oleh pengguna yang memiliki Username dan Password yang benar. Fitur ini memberi batas maksimal 3 kali percobaan gagal sebelum program tertutup otomatis.

```
while (percobaanSisa > 0 && !isLogin) {
        cout << "\nUsername (Nama) : ";
        getline(cin, inputNama);
        cout << "Password (NIM)  : ";
        getline(cin, inputNim);

        if (inputNama == penggunaSah.Username && inputNim ==
penggunaSah.Password) {
            isLogin = true;
            cout << ">>> Login Sukses! Selamat datang, " <<
penggunaSah.Username << ". <<<\n";
        } else {
            percobaanSisa--;
            cout << ">>> Username atau Password salah! <<<\n";
```

```
        }
    }
```

B. Fitur Tambah Data (Create)

Fungsi: Memasukkan data penyewaan powerbank yang baru (Nama, Merek, Durasi) ke dalam memori (*Array*). Fitur ini otomatis membuat ID Sewa secara berurutan agar tidak ada ID yang ganda.

```
void TambahData(Penyewaan dataSewa[], int &jumlahData, int
&nextId) {
    if (jumlahData < MAKS_DATA) {
        dataSewa[jumlahData].IdSewa = nextId;
        nextId++;
        jumlahData++;
        cout << ">>> Data berhasil ditambahkan! <<<\n";
    } else {
        cout << "Kapasitas data sudah penuh!\n";
    }
}
```

C. Fitur Lihat Seluruh Data (Read)

Fungsi: Menampilkan semua riwayat penyewaan powerbank yang telah diinput sebelumnya ke layar agar mudah dibaca.

```
void LihatData(Penyewaan dataSewa[], int jumlahData) {
    if (jumlahData == 0) {
        cout << "\nBelum ada data penyewaan.\n";
        return;
    }
    for (int i = 0; i < jumlahData; i++) {
    }
}
```

D. Fitur Cari Data (Search Rekursif)

Fungsi: Mencari lokasi baris (*index*) dari sebuah ID Sewa tertentu. Alih-alih menggunakan *looping* biasa, fungsi ini memanggil dirinya sendiri (rekursif) untuk mengecek satu per satu hingga data ditemukan atau data habis.

```
int CariDataRekursif(Penyewaan dataSewa[], int jumlahData, int
idTarget, int indexSekarang) {
    if (indexSekarang >= jumlahData) {
        return -1;
    }
    if (dataSewa[indexSekarang].IdSewa == idTarget) {
        return indexSekarang;
    }
    return CariDataRekursif(dataSewa, jumlahData, idTarget,
indexSekarang + 1);
}
```

E. Fitur Lihat Detail 1 Data (Read dengan Overloading)

Fungsi: Ini adalah contoh *Overloading Function* (nama fungsinya sama persis dengan Lihat Seluruh Data, namun jumlah parameternya berbeda). Fungsinya khusus untuk menampilkan detail dari satu data saja berdasarkan *index* yang didapat dari fitur Cari Data.

```
void LihatData(Penyewaan dataSewa[], int jumlahData, int
indexTarget) {
    cout << "\n=== DETAIL DATA DITEMUKAN ===\n";
    cout << "ID Sewa         : " << dataSewa[indexTarget].IdSewa
<< "\n";
    // ... (menampilkan atribut data lainnya secara spesifik)
}
```

F. Fitur Ubah Data (Update)

Fungsi: Memperbarui data lama (seperti jika penyewa ingin ganti merek powerbank atau nambah durasi sewa). Fitur ini akan mencari ID target, lalu menimpa data lamanya dengan ketikan data yang baru.

```
void UbahData(Penyewaan dataSewa[], int jumlahData) {
    // ... (meminta input idTarget)
    int indexDitemukan = CariDataRekursif(dataSewa, jumlahData,
idTarget, 0);

    if (indexDitemukan != -1) {
        // ... (input ulang NamaPenyewa, MerekPowerbank, DurasiJam
menggunakan indexDitemukan)
        cout << ">>> Data berhasil diubah! <<<\n";
    } else {
        cout << ">>> Data dengan ID tersebut tidak ditemukan!
<<<\n";
    }
}
```

G. Fitur Hapus Data (Delete)

Fungsi: Menghapus data penyewaan berdasarkan ID. Cara menghapusnya adalah dengan menemukan *index* datanya, lalu menggeser semua data yang ada di bawah *index* tersebut naik satu langkah ke atas, dan mengurangi `jumlahData` sebesar 1.

```
void HapusData(Penyewaan dataSewa[], int &jumlahData) {
    // ... (meminta input idTarget)
    int indexDitemukan = CariDataRekursif(dataSewa, jumlahData,
idTarget, 0);

    if (indexDitemukan != -1) {
        for (int j = indexDitemukan; j < jumlahData - 1; j++) {
            dataSewa[j] = dataSewa[j + 1]; // Menggeser array ke
atas (menimpa data yang mau dihapus)
        }
        jumlahData--;
        cout << ">>> Data berhasil dihapus! <<<\n";
    } else {
        // ... (data tidak ditemukan)
    }
}
```

H. Fitur Pengurutan Bubble Sort (Nama - Descending)

Fungsi: Mengurutkan data berdasarkan **Nama Penyewa** dari abjad Z ke A (Descending). Algoritma ini bekerja dengan membandingkan dua nama yang bersebelahan, lalu menukarnya jika posisinya salah.

```
void SortNamaDescending(Penyewaan dataSewa[], int jumlahData) {
    for (int i = 0; i < jumlahData - 1; i++) {
        for (int j = 0; j < jumlahData - i - 1; j++) {
            if (dataSewa[j].NamaPenyewa < dataSewa[j +
1].NamaPenyewa) {
                Penyewaan temp = dataSewa[j];
                dataSewa[j] = dataSewa[j + 1];
                dataSewa[j + 1] = temp;

                dataSewa[j].ptrDurasi = &dataSewa[j].DurasiJam;
                dataSewa[j + 1].ptrDurasi = &dataSewa[j +
1].DurasiJam;
            }
        }
    }
    cout << ">>> Data berhasil diurutkan berdasarkan Nama
(Descending) dengan Bubble Sort! <<<\n";
}
```

I. Fitur Pengurutan Selection Sort (Durasi Jam - Ascending)

Fungsi: Mengurutkan data berdasarkan **Durasi Sewa** dari yang paling sebentar ke paling lama (Ascending). Algoritma ini bekerja dengan mencari nilai durasi paling kecil dari kumpulan data, lalu meletakkannya di posisi teratas.

```
void SortDurasiAscending(Penyewaan dataSewa[], int jumlahData) {
    for (int i = 0; i < jumlahData - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < jumlahData; j++) {
            if (dataSewa[j].DurasiJam <
dataSewa[minIndex].DurasiJam) {
                minIndex = j;
            }
        }
```

```
        if (minIndex != i) {
            Penyewaan temp = dataSewa[i];
            dataSewa[i] = dataSewa[minIndex];
            dataSewa[minIndex] = temp;

            dataSewa[i].ptrDurasi = &dataSewa[i].DurasiJam;
            dataSewa[minIndex].ptrDurasi =
&dataSewa[minIndex].DurasiJam;
        }
    }
    cout << ">>> Data berhasil diurutkan berdasarkan Durasi Sewa
(Ascending) dengan Selection Sort! <<<\n";
}
```

J. Fitur Pengurutan Insertion Sort (ID Sewa - Ascending)

Fungsi: Ini adalah metode *sorting* bebas yang ditambahkan untuk mengurutkan **ID Sewa** dari kecil ke besar (Ascending). Sangat berguna untuk mengembalikan urutan data seperti semula. Algoritma ini bekerja seperti saat kita menyusun kartu remi di tangan.

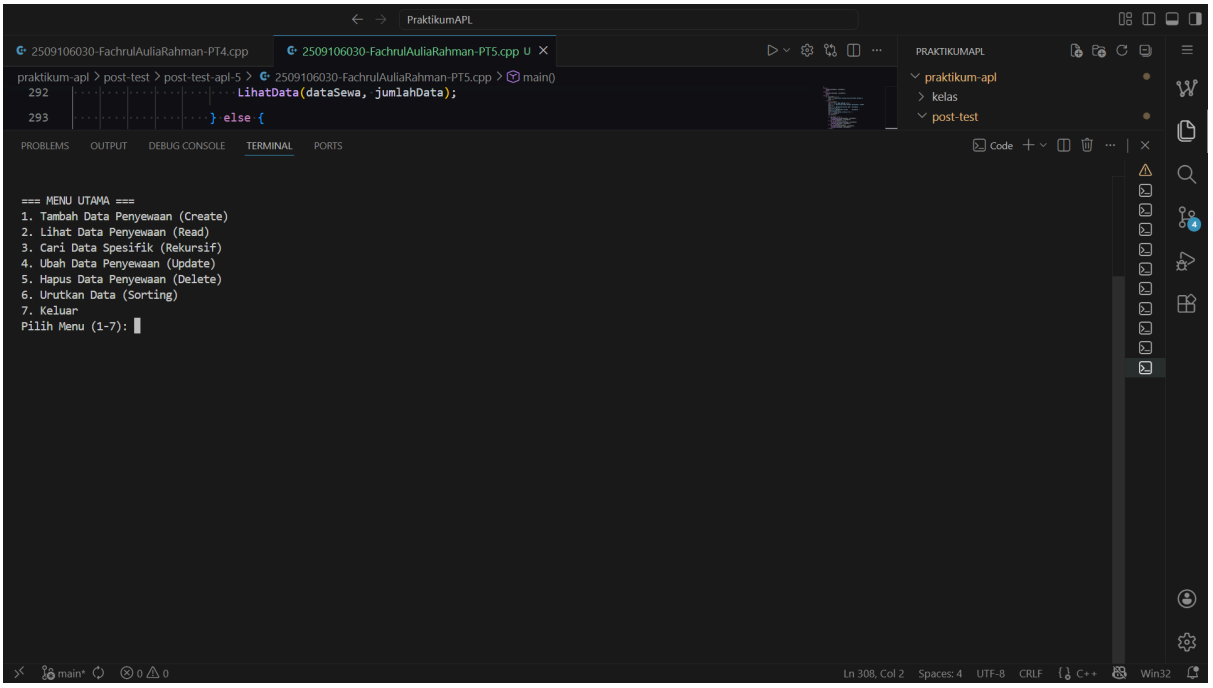
```
void SortIdAscending(Penyewaan dataSewa[], int jumlahData) {
    for (int i = 1; i < jumlahData; i++) {
        Penyewaan key = dataSewa[i];
        int j = i - 1;
        while (j >= 0 && dataSewa[j].IdSewa > key.IdSewa) {
            dataSewa[j + 1] = dataSewa[j];
            dataSewa[j + 1].ptrDurasi = &dataSewa[j +
1].DurasiJam;
            j = j - 1;
        }
        dataSewa[j + 1] = key;
        dataSewa[j + 1].ptrDurasi = &dataSewa[j + 1].DurasiJam;
    }
    cout << ">>> Data berhasil dikembalikan sesuai urutan ID
(Ascending) dengan Insertion Sort! <<<\n";
}
```

K. Sub-Menu Sorting (Di dalam Fungsi `main`)

Fungsi: Menu di `main()` (Pilihan 6) yang mencegah pengurutan jika data kurang dari 2, dan memberikan sub-menu agar pengguna bisa memilih ingin mengurutkan data menggunakan metode yang mana.

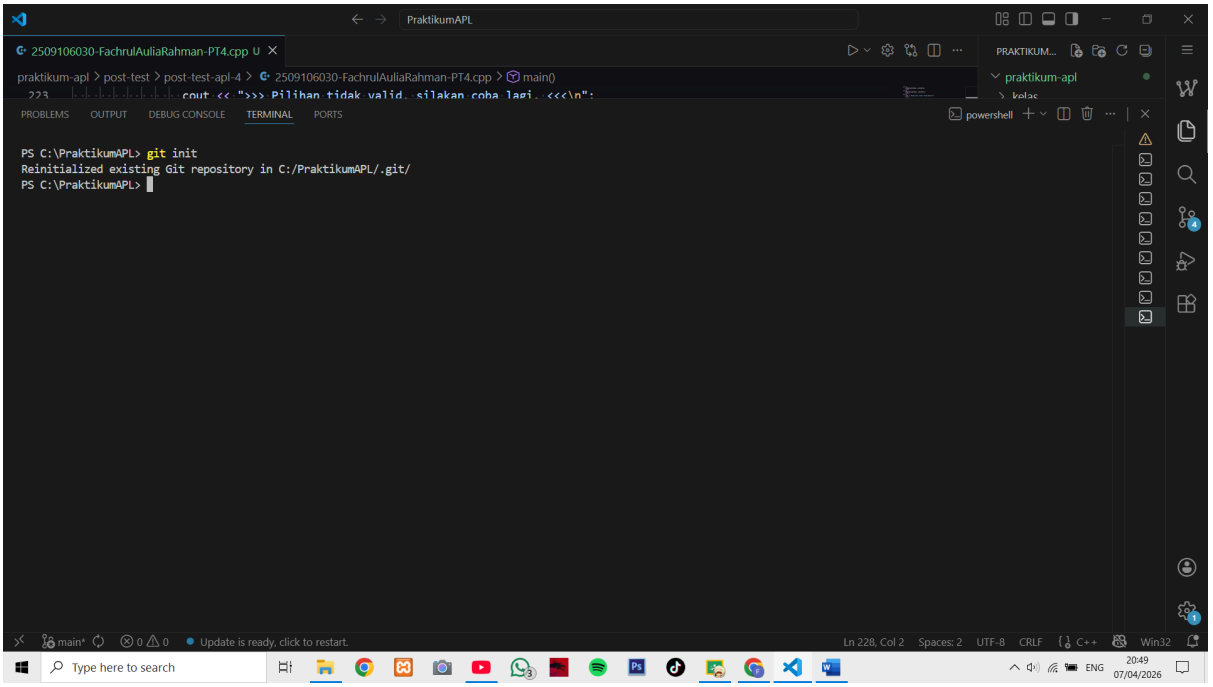
```
case 6: {
                if (jumlahData < 2) {
                    cout << "\nData masih kurang untuk diurutkan
(minimal 2 data).\n";
                } else {
                    int pilihSort;
                    cout << "\n=== MENU SORTING ===\n";
                    cout << "1. Berdasarkan Nama Penyewa
(Descending - Bubble Sort)\n";
                    cout << "2. Berdasarkan Durasi Sewa
(Ascending  - Selection Sort)\n";
                    cout << "3. Berdasarkan ID Sewa
(Ascending  - Insertion Sort)\n";
                    cout << "Pilih metode sorting (1-3): ";
                    cin >> pilihSort;
                    cin.ignore();

                    if (pilihSort == 1) {
                        SortNamaDescending(dataSewa, jumlahData);
                        LihatData(dataSewa, jumlahData);
                    } else if (pilihSort == 2) {
                        SortDurasiAscending(dataSewa, jumlahData);
                        LihatData(dataSewa, jumlahData);
                    } else if (pilihSort == 3) {
                        SortIdAscending(dataSewa, jumlahData);
                        LihatData(dataSewa, jumlahData);
                    } else {
                        cout << ">>> Pilihan sorting tidak valid!
<<<\n";
                    }
                }
                break;
            }
```

4. Hasil Output

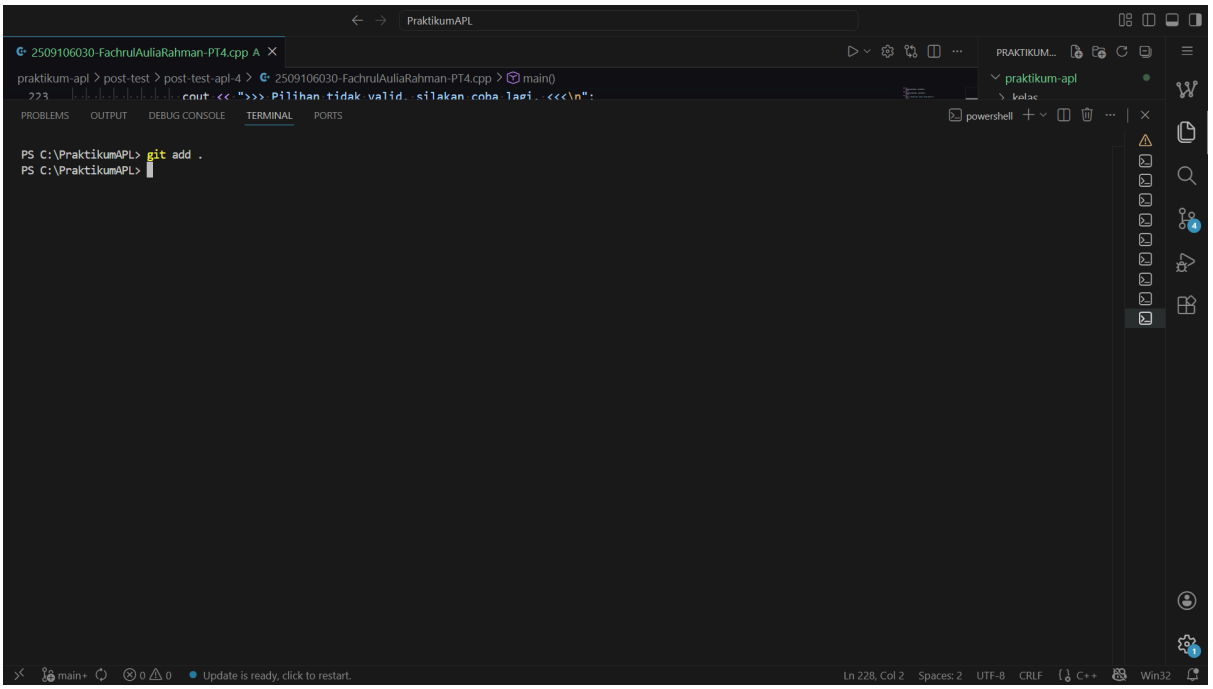


5. Langkah-langkah GIT

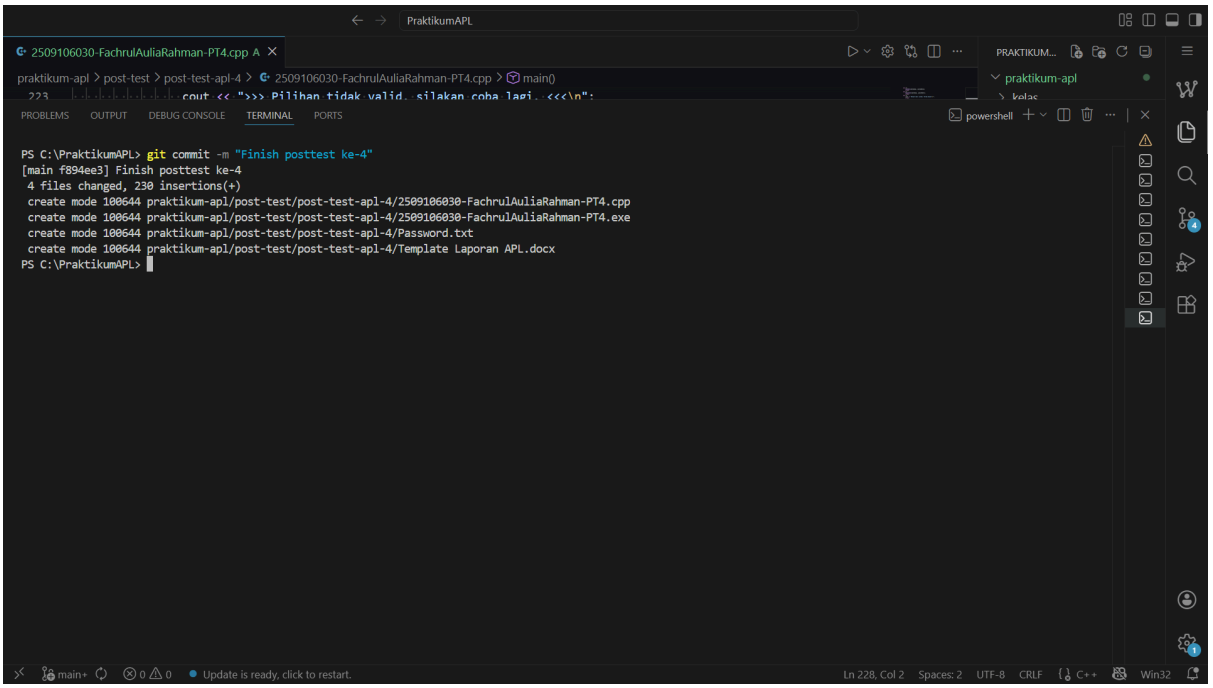
5.1 GIT init



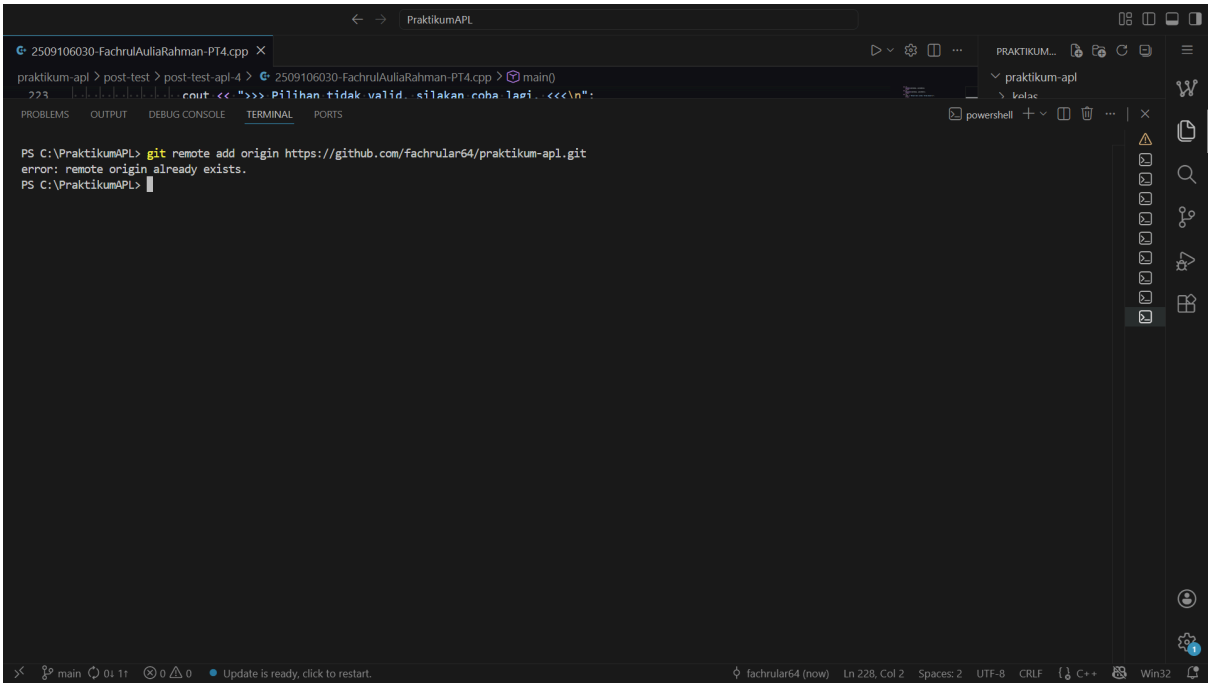
5.2 GIT add



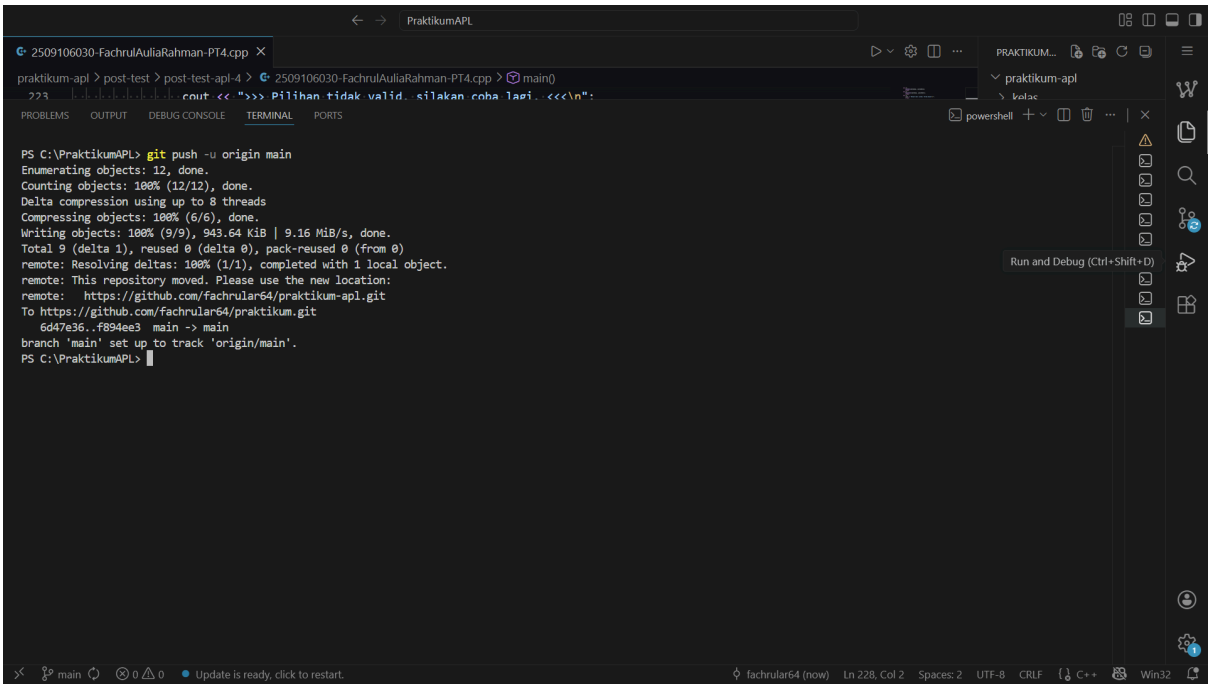
5.3 GIT commit



5.4 GIT remote



5.5 GIT push


PraktikumAPL
2509106030-FachrulAuliaRahman-PT4.cpp
praktikum-apl > post-test > post-test-apl-4 > 2509106030-FachrulAuliaRahman-PT4.cpp > main()
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
PS C:\PraktikumAPL> git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 943.64 KiB | 9.16 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote: https://github.com/fachrular64/praktikum-apl.git
To https://github.com/fachrular64/praktikum.git
6d47e36..f894ee3 main -> main
branch 'main' set up to track 'origin/main'.
PS C:\PraktikumAPL>
Run and Debug (Ctrl+Shift+D)
praktikum-apl
main
Update is ready, click to restart.
fachrular64 (now)
Ln 228, Col 2
Spaces: 2
UTF-8
CRLF
C++
Win32